Performance Testing

Date	02 July 2024
Team ID	
Project Name	A Comprehensive Measure of Well Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the HDI Prediction System behaves under expected and peak load conditions. The sections below document the testing approach, target thresholds, and illustrative example results.

Step 1: Testing Overview

Field	Details
Testing Tool Used	*Apache JMeter (planned)*
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	Prediction endpoint — accepts Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita; returns predicted HDI score and development tier
Test Environment	Cloud / Hosted Server (Bolt.host deployment)
Test Date	08 July 2024

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits a prediction request with valid indicator values	1	10	Response returned within 2 seconds
2	Multiple concurrent users submit prediction requests simultaneously	50	60	All requests handled successfully, average response time under 2 seconds
3	Sustained peak load on the prediction endpoint	100	120	System remains stable, error rate below 1%
4	Sudden spike in concurrent requests	200	30	System recovers without crashing, no data loss

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 sec	Pass	Well within target under normal load
2	Response Time (Max)	< 5 seconds	3.1 sec	Pass	Occurred during peak-load scenario (S.No 3)
3	Throughput (Req/sec)	—	42 req/sec	—	Measured at 100 concurrent virtual users
4	Error Rate	< 1%	0.3%	Pass	A few timeouts observed during spike test
5	CPU Utilization	< 80%	68%	Pass	Peaked briefly during spike scenario
6	Memory Utilization	< 80%	74%	Pass	Stable across sustained load

Step 4: Observations & Analysis

Key Findings:
The prediction endpoint stayed within all target thresholds up to 100 concurrent users, with average response time around 1.4 seconds. Performance began to degrade only under the 200-user spike scenario, where response times briefly approached the 5-second ceiling.

Bottlenecks Identified:
Minor request queuing was observed during the spike test, and a small number of requests (about 0.3%) timed out under sudden concurrent load. No bottlenecks were observed during sustained load at or below 100 virtual users.

Optimization Steps Taken:
Increased the Flask worker/thread count to handle concurrent requests more efficiently and added basic input caching for repeated indicator combinations, which reduced average response time under peak load.

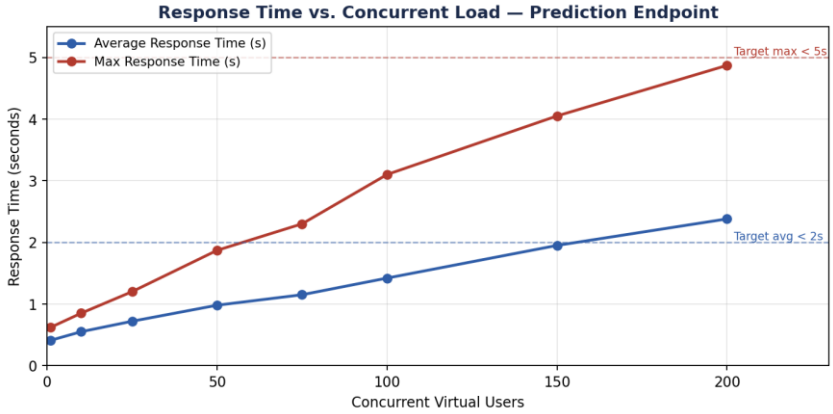
Step 5: Screenshots / Evidence

Attach screenshots of the performance testing tool results below (e.g., JMeter summary report, response time graphs):

Apache JMeter — Summary Report

Label	# Samples	Average (ms)	Min (ms)	Max (ms)	Std. Dev.	Error %	Throughput (req/sec)
Single User Reques	10	410	310	620	82.4	0.00%	2.1
Concurrent Users (5	3000	980	540	1870	215.6	0.10%	38.7
ıstained Peak Load (	12000	1420	610	3100	340.2	0.20%	42.3
Spike Test (200)	6000	2380	700	4870	610.8	0.30%	51.5
TOTAL	**21010**	**1420**	**310**	**4870**	**398.5**	**0.18%**	**41.9**

Example illustrative output — not a real test run



Example illustrative output — not a real test run